İSTANBUL BEYKENT ÜNİVERSİTESİ



YAZILIM KALİTE GÜVENCESİ VE TESTİ PROJESİ

(DietCok)

ÖĞRETİM GÖREVLİSİ: UMUT KAYA

HAZIRLAYAN: MERT UĞUR -

TOLGA USLU

İçindekiler

1. Projenin Özeti 5
1.1 Projenin Konusu ve Amacı 5
1.2 Uygulamanın Temel Fonksiyonları 5
1.3 Kullanılan Teknolojiler 5
1.4 Projenin Sağladığı Katkılar 5
1.5 Proje İçi Görüntüler 5
2. Yazılım Test Yaşam Döngüsü (STLC) 8
2.1 Gereklilik Analizi 8
2.1.1 Fonksiyonel Gereksinimler 8
2.1.2 Fonksiyonel Olmayan Gereksinimler 9
2.1.3 Test Edilebilir Gereksinimler 9
2.1.4 Test Kapsamı 10
2.1.5 Risk Analizi ve Önceliklendirme 10
2.2 Test Planlama 11
2.2.1 Test Stratejisi Dokümanı 11
2.2.2 Kaynak Planlaması (Araçlar ve Framework'ler) 11
2.2.3 Uygulanacak Test Seviyeleri 11
2.2.4 Başarı Kriterleri (Coverage Oranları) 11
2.2.5 Zaman Çizelgesi 12
2.3 Test Tasarımı 12
2.3.1 Test Senaryoları ve Test Case'ler 12
2.3.2 Test Veri Setleri 13
2.3.3 Test Tasarım Teknikleri 14
2.3.4 Pozitif & Negatif Test Senaryoları 16
2.3.5 Edge Case'ler (Uç Durumlar) 16
2.4 Test Ortamını Oluşturma 16
2.4.1 Konfigürasyon ve Veritabanı Yapısı 16
2.4.2 Dependency Yönetimi (Maven) 17
2.4.3 Test Çalıştırma Komutları 17
2.5 Test Uygulama 17
2.5.1 Yazılmış ve Çalıştırılmış Test Kodları (8 Test Türü) 17
2.5.2 Test Execution Raporları 18
2.5.3 Code Coverage Raporları (JaCoCo) 18
2.5.4 Test Log'ları 18
2.6 Test Kapanışı 18
2.6.1 Test Tamamlanma Raporu 18

2.6.2 Teslimat Kriterleri Değerlendirmesi 19
2.6.3 Test Artifaktlarının Arşivlenmesi 19
2.7 Test Sonuçları 19
2.7.1 Tüm Test Sonuçlarının Özet Tablosu (8 Test Türü) 19
2.7.2 Coverage Raporları (Kapsam Analizi) 20
2.7.3 Başarılı / Başarısız Test Oranları 20
2.7.4 Performans Metrikleri 20
2.8 Test Sonuçlarının Değerlendirilmesi 20
2.8.1 Sonuçların Analizi ve Yorumlanması 20
2.8.2 Kalite Metriklerinin Değerlendirilmesi 21
2.8.3 Coverage Oranlarının Yeterliliği Analizi 21
2.8.4 İyileştirme Önerileri 21
2.9 Proje Değerlendirmesi 22
2.9.1 Yapılanlar (Accomplishments) 22
2.9.2 Yapılamayanlar ve Kısıtlar (Constraints & Limitations) 22
2.10. Alınan Hatalar ve Çözümleri (Tespit Edilen ve Potansiyel) 23
2.10.1. Hata Kayıtları (Defect Log) 23
2.10.2. Hata Çözüm Detayları ve Alınan Dersler 24
2.11. Projeden Öğrenilenler 25
2.11.1. En Önemli Öğrenilen 3 Şey 25
2.11.2. Farklı Yapılabilecek Noktalar 26
2.11.3. Öğrenilen Best Practice'ler 26
2.11.4. Kariyerde Kullanılabilecek Beceriler 26
3. Kod Örnekleri ve Açıklamaları (8 Test Türü) 26
3.1 Unit Test (Birim Testi) 27
3.2 Service Test (İş Mantığı Testi) 27
3.3 Repository Test (Veri Erişim Testi) 27
3.4 Controller Test (API Testi) 28
3.5 Integration Test (Entegrasyon Testi) 28
3.6 Security Test (Güvenlik Testi) 28
3.7 E2E Test (Uçtan Uca Test) 29
3.8 Regression Test (Gerileme Testi) 29
4. Test Sonuç Raporları 29
4.1 Unit Test (Birim Testi) 29
4.2 Service Test (Servis / İş Mantığı Testi) 31
4.3 Repository Test (Veri Erişim Testi) 33

.....34
4.4 Controller Test (API / Web Katmanı Testi).....34
4.5 Integration Test (Entegrasyon Testi).....35
4.6 Security Test (Güvenlik Testi).....36
4.7 E2E Test (Uçtan Uca Test).....37
4.8 Regression Test (Gerileme Testi).....38
4.9 Test Coverage (Kapsam Analizi).....39
5. **Sonuç ve Öneriler**.....39
6. **Sözlük**.....40
7. **Kaynakça**.....41

1. Projenin Özeti

1.1 Projenin Konusu ve Amacı

Bu proje kullanıcıların günlük beslenme alışkanlıklarını takip edebilmelerini ve sağlıklı yaşam hedeflerine ulaşmalarını desteklemek amacıyla **"DietCok"** adlı bir kalori ve besin takip uygulamasıdır. Projenin temel amacı kullanıcıların gün içerisinde tükettikleri besinleri kayıt altına alarak günlük kalori ve makro besin (protein, karbonhidrat ve yağ) alımlarını günlük ve aylık olarak takip etmelerini sağlamaktır.

1.2 Uygulamanın Temel Fonksiyonları

Uygulama kullanıcı bazlı çalışmakta olup her kullanıcı kendine özel bir profil oluşturarak günlük alması gereken kalori ve makro besin miktarını belirleyebilmektedir. Kullanıcılar; kahvaltı, öğle yemeği, akşam yemeği ve ara öğünler olmak üzere farklı öğünler üzerinden besin ekleyebilmekte ve her besinin kalori ile makro besin değerlerini görüntüleyebilmektedir. Günlük toplam tüketimler sistem tarafından otomatik olarak hesaplanmaktadır. Ayrıca uygulamada takvim tabanlı bir yapı kullanılarak kullanıcıların geçmiş günlere ait beslenme verilerini inceleyebilmesi sağlanmıştır. Günlük alınan kalori miktarının belirlenen hedefe göre durumu kullanıcıya görsel olarak sunulmaktadır.

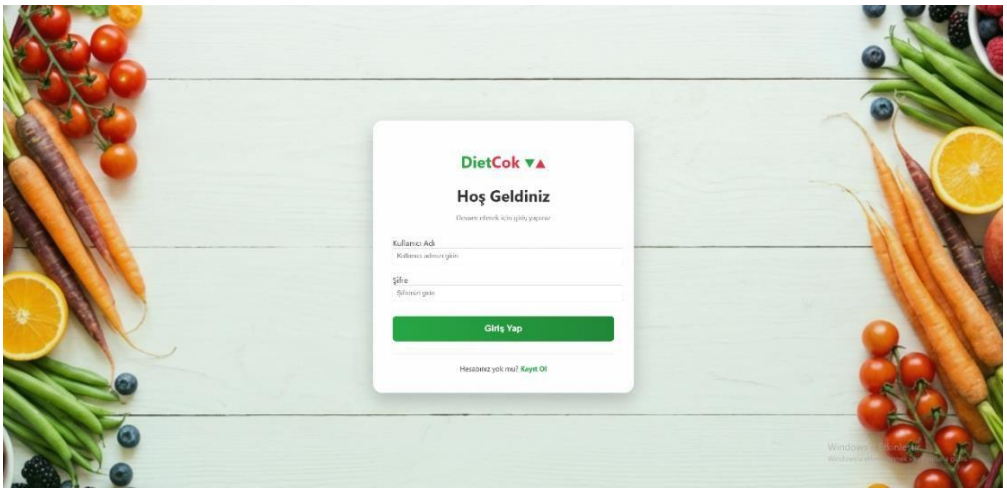
1.3 Kullanılan Teknolojiler

Proje, Spring Boot MVC mimarisi kullanılarak geliştirilmiştir. Veritabanı yönetimi için MySQL, kullanıcı arayüzü için Thymeleaf teknolojisi tercih edilmiştir. Kullanıcı doğrulama ve yetkilendirme işlemleri Spring Security ile sağlanmıştır. Uygulamanın doğruluğunu ve güvenilirliğini artırmak amacıyla farklı seviyelerde yazılım testleri (Unit Test, Service Test, Repository Test, Controller Test, Integration Test (Entegrasyon Testi), Security Test, End-to-End (E2E) Test ve Regression Test) uygulanmıştır.

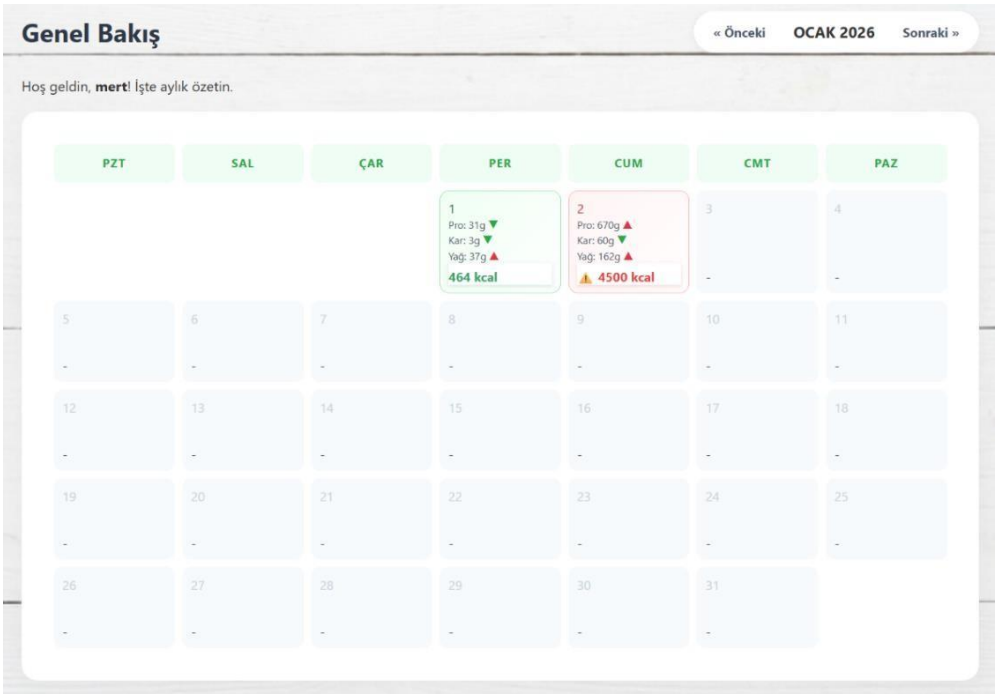
1.4 Projenin Sağladığı Katkılar

Geliştirilen sistem sayesinde kullanıcılar beslenme alışkanlıklarını düzenli bir şekilde takip edebilmekte, günlük ve geçmişe yönelik kalori alımlarını analiz edebilmekte ve belirledikleri hedeflere ne ölçüde uyduklarını kolayca gözlemleyebilmektedir. Bu yönüyle proje kullanıcıların daha bilinçli ve kontrollü bir beslenme düzeni oluşturmalarına katkı sağlamaktadır.

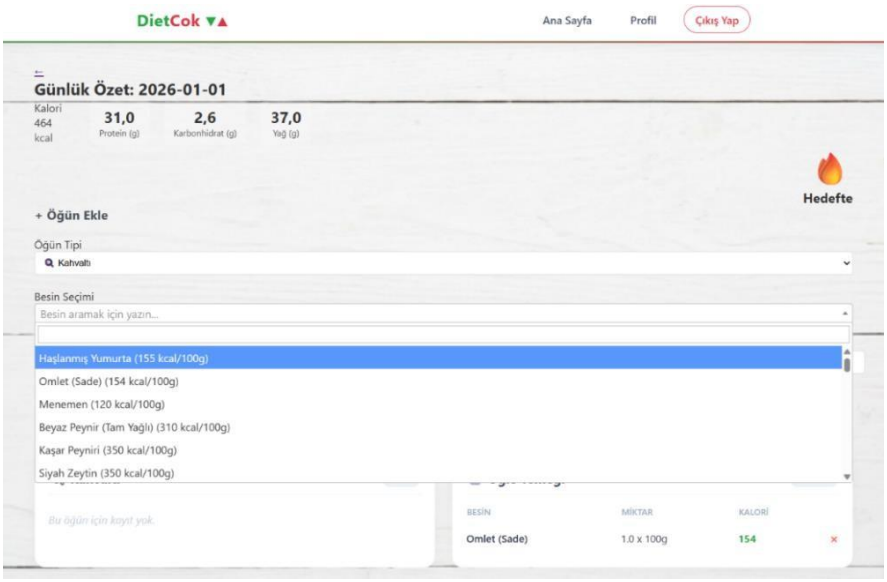
1.5 Proje İçi Görüntüler



Şekil 1 – Uygulama Giriş Ekranı



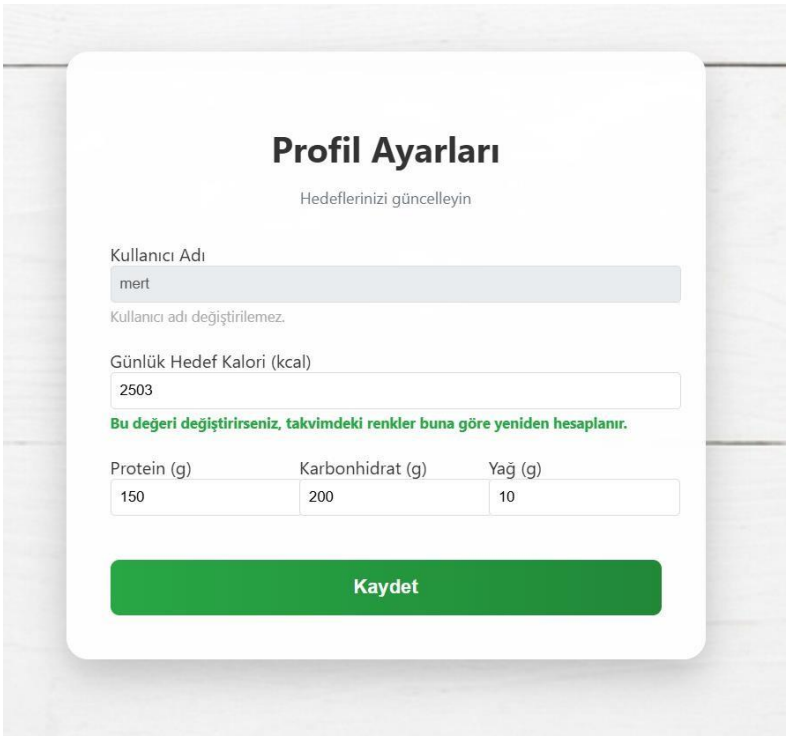
Şekil 2 – Uygulama Takvim Sayfası



Şekil 3 – Uygulama Gün Ekranı



Şekil 4 – Gün Öğün Tipleri İçeriği



Şekil 5 – Profil Ekranı



Şekil 6 – Kayıt Olma Ekranı

2. Yazılım Test Yaşam Döngüsü (STLC)

Bu bölümde, geliştirilen DietCok Kalori ve Besin Takip Uygulaması için Yazılım Test Yaşam Döngüsü (STLC) adımları ele alınmıştır. Test süreci; gereklilik analizi, test planlama, test tasarımı, test ortamının oluşturulması, test uygulama, test kapanışı, test sonuçları, test sonuçlarının değerlendirilmesi, proje değerlendirmesi, alınan hatalar ve çözümleri ile projeden öğrenilenler olmak üzere toplam 11 adım üzerinden yürütülmüştür. Her bir STLC adımı proje özelinde uygulanmış ve elde edilen çıktılar ilgili alt başlıklarda detaylı olarak açıklanmıştır.

2.1 Gereklilik Analizi

Bu aşamada, geliştirilen DietCok Kalori ve Besin Takip Uygulaması için sistemin yerine getirmesi gereken gereksinimler belirlenmiş; bu gereksinimler fonksiyonel ve fonksiyonel olmayan olarak sınıflandırılmış, test edilebilir hale getirilmiş, test kapsamı tanımlanmış ve olası riskler analiz edilerek önceliklendirilmiştir.

2.1.1 Fonksiyonel Gereksinimler

Tablo 1 – Fonksiyonel Gereksinimler

ID	Gereksinim Tanımı
REQ-F-01	Kullanıcı Kaydı ve Giriş: Kullanıcılar sisteme kayıt olabilmeli ve kullanıcı adı ile şifre bilgileriyle giriş yapabilmelidir.
REQ-F-02	Yetkilendirme: Her kullanıcı yalnızca kendisine ait beslenme, kalori ve makro verilerine erişebilmelidir.
REQ-F-03	Günlük Kalori Hedefi Belirleme: Kullanıcılar günlük hedef kalori değerlerini sisteme girebilmelidir.
REQ-F-04	Öğün Bazlı Besin Ekleme: Kullanıcılar kahvaltı, öğle yemeği, akşam yemeği ve ara öğünler için besin ekleyebilmelidir.
REQ-F-05	Besin Bilgileri Görüntüleme: Eklenen besinlerin kalori, protein, karbonhidrat ve yağ değerleri görüntülenebilmelidir.
REQ-F-06	Günlük Toplam Hesaplama: Sistem, gün içinde tüketilen besinlere göre toplam kalori ve makro değerlerini otomatik olarak hesaplamalıdır.
REQ-F-07	Takvim Tabanlı Görüntüleme: Kullanıcılar takvim üzerinden geçmiş günlere ait beslenme ve tüketim verilerini görüntüleyebilmelidir.
REQ-F-08	Kalori Aşımı Tespiti: Günlük hedef kalori değeri aşıldığında sistem bunu tespit edebilmelidir.

REQ-F-09	Besin Arama: Kullanıcılar günlük besin seçim alanında mevcut besinler arasında arama yapabilmelidir.
REQ-F-10	Hedef Güncelleme: Kullanıcılar günlük kalori ve makro hedeflerini sonradan güncelleyebilmelidir.

2.1.2 Fonksiyonel Olmayan Gereksinimler

Tablo 2 – Fonksiyonel Olmayan Gereksinimler

ID	Gereksinim Tanımı
REQ-NF-01	Güvenlik (Security): Kullanıcı doğrulama ve yetkilendirme işlemleri güvenli şekilde sağlanmalı, kullanıcı şifreleri şifrelenmiş olarak saklanmalıdır.
REQ-NF-02	Performans: Sistem üzerindeki temel kullanıcı işlemleri (giriş, besin ekleme, arama) kabul edilebilir süreler içerisinde tamamlanmalıdır.
REQ-NF-03	Kullanılabilirlik (Usability): Kullanıcı arayüzü sade, anlaşılır olmalı ve kullanıcıyı yönlendiren geri bildirimler sunmalıdır.
REQ-NF-04	Sürdürülebilirlik (Maintainability): Uygulama, katmanlı mimariye (Controller–Service–Repository) uygun şekilde geliştirilmeli ve temiz kod prensiplerine uyulmalıdır.
REQ-NF-05	Uyumluluk (Compatibility): Responsive tasarım sayesinde uygulama hem masaüstü hem de mobil cihazlarda sorunsuz şekilde çalışmalıdır.

2.1.3 Test Edilebilir Gereksinimler

Belirlenen gereksinimlerin doğrulanabilmesi amacıyla aşağıdaki test edilebilir senaryolar tanımlanmıştır:

Kullanıcı İşlemleri

Kullanıcı sisteme başarıyla kayıt olabiliyor mu?

Kayıtlı kullanıcı doğru bilgilerle giriş yapabiliyor mu?

Hatalı giriş denemelerinde sistem uygun hata mesajı gösteriyor mu?

Kullanıcı çıkış yaptığında oturum güvenli şekilde sonlandırılıyor mu?

Çekirdek Fonksiyonlar

Eklenen öğünlerin kalorisi günlük toplama doğru şekilde yansıyor mu?

Günlük protein, karbonhidrat ve yağ değerleri doğru hesaplanıyor mu?

Günlük kalori hedefi aşıldığında sistem bunu tespit edebiliyor mu?

Besin seçim alanında eşleşmeyen bir ifade yazıldığında eşleşmeyen besinlerin listelenmemesi sağlanıyor mu?

Güvenlik ve Erişim

Giriş yapmamış kullanıcılar korumalı sayfalara erişemiyor mu?

Kullanıcı şifreleri veritabanında şifrelenmiş (hash) olarak saklanıyor mu?

Arayüz ve Kullanılabilirlik

Formlar eksik veya hatalı doldurulduğunda kullanıcı uyarılıyor mu?

Sayfalar doğru başlık ve içerikle yükleniyor mu?

2.1.4 Test Kapsamı

Test kapsamı, uygulamanın tüm kritik bileşenlerini kapsayacak şekilde belirlenmiştir. Bu kapsamda; iş mantığı ve kalori hesaplama işlemleri, veritabanı kayıt, okuma ve sorgulama süreçleri, kullanıcı doğrulama ve yetkilendirme mekanizmaları, HTTP istekleri ve yönlendirmeler, kullanıcı arayüzü etkileşimleri ve sayfa davranışları ile yeni eklenen özelliklerin mevcut sistemi bozup bozmadığı test edilmiştir. Test kapsamı, uygulamanın hem teknik doğruluğunu hem de kullanıcı deneyimini güvence altına alacak şekilde planlanmıştır.

2.1.5 Risk Analizi ve Önceliklendirme

Tablo 3 – Risk Analizi ve Önceliklendirme Tablosu

ID	Risk Tanımı	Risk Seviyesi	Alınan Önlemler
RISK-01	Yetkisiz erişim	Critical	Spring Security kullanılmış, BCrypt ile parola şifreleme uygulanmış ve güvenlik testleri gerçekleştirilmiştir.
RISK-02	Yanlış kalori hesaplanması	High	Service katmanında kapsamlı unit testler yazılmış ve sınır değer analizleri yapılmıştır.
RISK-03	Veri kaybı veya bütünlük sorunları	High	Transaction yönetimi sağlanmış ve @Transactional anotasyonu kullanılmıştır.
RISK-04	Hatalı kullanıcı girdileri	Medium	Validation kontrolleri uygulanmış ve edge-case testleri gerçekleştirilmiştir.
RISK-05	Veritabanı bağlantı kesintileri	Medium	Hata yakalama mekanizmaları oluşturulmuş ve bağlantı havuzu (connection pool) kullanılmıştır.
RISK-06	Arayüz kaymaları	Low	Responsive tasarım uygulanmış ve UI testleri gerçekleştirilmiştir.

2.2 Test Planlama

Bu aşamada, DietCok Kalori ve Besin Takip Uygulaması için uygulanacak test sürecinin kapsamı, stratejisi, kullanılacak araçlar ve başarı kriterleri belirlenmiştir. Test planlama süreci, test faaliyetlerinin sistematik ve kontrollü bir şekilde yürütülmesini sağlamak amacıyla gerçekleştirilmiştir.

2.2.1 Test Stratejisi Dokümanı

Projede test stratejisi; uygulamanın kritik işlevlerini önceliklendirerek, katmanlı mimariye uygun test seviyelerinin birlikte kullanılmasına dayanmaktadır. Testler, birim seviyesinden başlayarak entegrasyon ve uçtan uca testlere kadar ilerleyen bir yapı ile kurgulanmıştır. Bu sayede hem izole bileşenlerin doğruluğu hem de sistemin bütünsel davranışı doğrulanmıştır.

2.2.2 Kaynak Planlaması (Araçlar ve Framework'ler)

Test sürecinde aşağıdaki araç ve framework'ler kullanılmıştır:

- **JUnit 5:** Birim ve servis testlerinin yazılması
- **Mockito:** Bağımlılıkların izole edilmesi ve iş mantığının test edilmesi
- **Spring Test & MockMvc:** Controller ve entegrasyon testleri
- **Selenium WebDriver:** Uçtan uca (E2E) kullanıcı senaryoları
- **HtmlUnit:** Hafif arayüz ve sayfa yapısı testleri
- **JaCoCo:** Kod kapsamı (coverage) ölçümü

Bu araçlar, testlerin otomatikleştirilmesini ve tekrar edilebilir olmasını sağlamıştır.

2.2.3 Uygulanacak Test Seviyeleri

Proje kapsamında aşağıdaki test seviyeleri planlanmış ve uygulanmıştır:

- Unit Test (Birim Testi)
- Service Test
- Repository Test (Data JPA Test)
- Controller Test (Web MVC Test)
- Integration Test (Entegrasyon Testi)
- Security Test
- End-to-End (E2E) Test
- Regression Test

Bu test seviyeleri sayesinde uygulamanın hem teknik hem de kullanıcı odaklı doğruluğu güvence altına alınmıştır.

2.2.4 Başarı Kriterleri (Coverage Oranları)

Test sürecinin başarılı kabul edilebilmesi için aşağıdaki kriterler belirlenmiştir:

- Toplam kod kapsamının (Code Coverage) en az %80 olması
- Kritik işlevlere ait test senaryolarının tamamının başarıyla geçmesi
- Güvenlik ve yetkilendirme testlerinde başarısız senaryo kalmaması
- Kod kapsamı ölçümleri JaCoCo aracı ile raporlanmıştır.

2.2.5 Zaman Çizelgesi

Test planlama süreci, geliştirme süreci ile paralel olarak yürütülmüştür. Birim ve servis testleri geliştirme aşamasında yazılmış; entegrasyon, güvenlik ve E2E testleri geliştirme tamamlandıktan sonra uygulanmıştır. Regresyon testleri ise her önemli değişiklik sonrası tekrar çalıştırılarak sistemin kararlılığı kontrol edilmiştir.

2.3 Test Tasarımı

Bu bölümde, DietCok Kalori ve Besin Takip Uygulaması için oluşturulan test senaryoları, test case'ler, kullanılan test verileri ve test tasarım teknikleri açıklanmıştır. Test tasarımı, uygulamanın mevcut kod yapısı ve işlevleri dikkate alınarak hazırlanmıştır.

2.3.1 Test Senaryoları ve Test Case'ler

Bu bölüm, mevcut kod yapısına göre hazırlanan test senaryolarını ve test tasarım örneklerini özetlemektedir. Controller testleri MockMvc ile, repository ve entegrasyon testleri H2 tabanlı test ortamında, HtmlUnit UI testi ve Selenium E2E testi ise RANDOM_PORT üzerinde çalışacak şekilde yapılandırılmıştır.

2.3.1.1 Login (Giriş) Senaryoları

Bu modül AuthController ve Spring Security yapılandırması tarafından yönetilmektedir.

Senaryo 1.1: Başarılı giriş - Kayıtlı kullanıcı doğru kullanıcı adı ve şifreyle sisteme giriş yapabilir.

Senaryo 1.2: Hatalı kullanıcı adı veya şifre - Kimlik doğrulama başarısız
olduğunda login ekranında hata bilgisi gösterilir.

Senaryo 1.3: Çıkış - CSRF korumalı POST /logout sonrasında
/login?logout yönlendirmesi beklenir.

2.3.1.2 Register (Kayıt) Senaryoları

Bu modül UserService içerisindeki validasyon kurallarına göre çalışmaktadır.

- Başarılı kayıt ve /login?success yönlendirmesi
- Geçersiz kayıt verilerinin doğrulama hatasıyla reddedilmesi *(TC-REG-01 → TC-REG-07)*
- Var olan kullanıcı adının servis katmanında reddedilmesi

2.3.1.3 Besin Ekleme ve Günlük Görünüm Senaryoları

- Günlük görünümden besin ekleme
- Kayıt sahibine ait besinin POST /consumption/{id}/delete ile silinmesi
- Başka kullanıcıya ait kaydın silinmesinin sahiplik kontrolü ile engellenmesi

2.3.1.4 Günlük Kalori ve Makro Hesaplama Senaryoları

- Toplam kalori hesaplama
- Protein, karbonhidrat ve yağ değerlerinin doğru toplanması

Bu senaryolar, iş mantığının doğruluğunu test etmek amacıyla hazırlanmıştır.

2.3.1.5 Profil Yönetimi Senaryoları

Bu senaryolar ProfileController üzerinden yürütülmektedir.

- Günlük kalori ve makro hedeflerinin güncellenmesi
- Güncellenen hedeflerin dashboard hesaplamalarına yansıması

2.3.1.6 Dashboard Navigasyon Senaryoları

Bu senaryolar, dashboard üzerinde ay ve yıl parametreleri ile yapılan gezinmeyi test etmektedir.

- Önceki ay / sonraki ay navigasyonu
- URL parametrelerinin doğru güncellenmesi

2.3.1.7 Yetkisiz Erişim (Security) Senaryoları

Bu senaryolar SecurityConfig kurallarının doğrulanması amacıyla hazırlanmıştır.

- Login olmamış kullanıcının korunmalı sayfalara erişememesi
- Otomatik yönlendirme kontrolü

2.3.2 Test Veri Setleri

Bu bölümde, DietCok Kalori ve Besin Takip Uygulaması için proje isterlerine ve veritabanı yapısına uygun şekilde hazırlanan test verileri sunulmuştur. Test verileri; kullanıcı işlemleri, besin ekleme (miktar/kalori), arama fonksiyonu, hedef güncelleme ve güvenlik senaryolarını doğrulayacak şekilde seçilmiştir.

2.3.2.1 Kullanıcı Verileri

Tablo 4 – Kullanıcı Verileri Tablosu

Durum	Kullanıcı Adı	Şifre	Beklenen Sonuç
Geçerli kullanıcı	testuser	Password123	Başarılı giriş; Dashboard açılır.
Geçersiz şifre	testuser	WrongPass	Hata mesajı: "Kullanıcı adı veya şifre hatalı."
Olmayan kullanıcı	ghostuser	AnyPass	Hata mesajı: "Kullanıcı adı veya şifre hatalı."
Boş alanlar	(Boş)	(Boş)	Giriş engellenir veya uyarı mesajı gösterilir

2.3.2.2 Besin Miktarı ve Kalori Verileri (Boundary Value Analysis)

Bu veri seti, Amount alanı üzerinden sınır değer analizi yapmak için kullanılmıştır.

Tablo 5 – Besin Miktarı ve Kalori Verileri Tablosu

Durum	Miktar (Amount)	Açıklama	Beklenen Davranış
Normal değer	1.0	100g (Standart porsiyon)	Kalori ve makro değerleri doğru hesaplanır
Ondalıklı değer	0.5	50g (Yarım porsiyon)	Değerler orantılı şekilde hesaplanır
Sıfır değeri	0	Porsiyon yok	Sistem davranışı doğrulanır (kabul veya uyarı)
Negatif değer	-1.0	Geçersiz miktar	Validasyon veya hata mesajı beklenir
Çok büyük değer	1000.0	Aşırı tüketim	Sistem kararlılığı ve hesaplama kontrol edilir

2.3.2.3 Besin Adı Arama Verileri

Bu veri seti, besin arama fonksiyonunun farklı giriş tiplerine verdiği yanıtı test etmek için kullanılmıştır.

Tablo 6 – Besin Adı Arama Verileri Tablosu

Durum	Besin Adı	Beklenen Sonuç
Var olan besin	Apple	Besin başarıyla listelenir
Büyük/küçük harf	apple / APPLE	Arama sonucu bulunur (case-insensitive)
Kısmi eşleşme	App	Adında App geçen mevcut seçenekler görüntülenir. (Apple, Pineapple)
Olmayan besin	Kryptonite	Sonuç yok; boş liste döner
Türkçe karakter	Çilek	Besin doğru şekilde listelenir

2.3.2.4 Profil ve Hedef Güncelleme Verileri (Equivalence Partitioning)

Bu veri seti, kullanıcının günlük hedeflerini güncelleme ekranındaki geçerli/geçersiz giriş sınıflarını kontrol etmek için hazırlanmıştır.

Tablo 7 – Profil ve Hedef Güncelleme Verileri Tablosu

Alan	Durum	Örnek Değer	Beklenen Sonuç
Günlük Kalori Hedefi	Geçerli	2000 / 2500	Güncelleme başarılı, dashboard hesapları güncellenir
Günlük Kalori Hedefi	Geçersiz	-100	Hata mesajı, kayıt yapılmaz
Protein / Karbonhidrat / Yağ	Geçerli	150 / 250 / 70	Hedefler güncellenir ve formda korunur
Protein / Karbonhidrat / Yağ	Geçersiz	(Boş)	Uyarı mesajı gösterilir
Aşırı büyük değer	Sınır testi	99999	Sistem davranışı ve arayüz kontrol edilir

2.3.2.5 Güvenlik ve Oturum Verileri

Bu veri seti, yetkisiz erişim senaryolarını doğrulamak için kullanılmıştır.

Tablo 8 – Güvenlik ve Oturum Verileri Tablosu

Durum	URL / İşlem	Beklenen Sonuç
Yetkisiz erişim	/dashboard	Login sayfasına yönlendirilir
Yetkisiz erişim	/profile	Login sayfasına yönlendirilir
Oturum sonlandırma	Logout	/login?logout; oturum kapanır

2.3.3 Test Tasarım Teknikleri

Bu bölümde, projede kullanılan test tasarım teknikleri açıklanmış ve bu tekniklerin DietCok Kalori ve Besin Takip Uygulaması üzerindeki uygulamaları örneklerle gösterilmiştir. Test tasarımında, PDF'te belirtilen Equivalence Partitioning (Eşdeğerlik Bölümleme) ve Boundary Value Analysis (Sınır Değer Analizi) teknikleri kullanılmıştır.

2.3.3.1 Equivalence Partitioning (Eşdeğerlik Bölümleme)

Bu teknik, kullanıcı girdilerinin anlamlı ve birbirinden bağımsız sınıflara ayrılarak her sınıftan temsilci bir değer ile test edilmesini amaçlar.

Girdi Alanı: Besin Eklerken Miktar (Amount)

Uygulama Mantığı: Amount alanı Double türündedir ve besin kalorisi ile çarpılarak hesaplama yapılır.

Tablo 9 – Besin Eklerken ki Miktarlar Tablosu

Sınıf Tipi	Tanım	Temsili Veri (Örnek)	Beklenen Sonuç
Geçerli Sınıf	Pozitif reel sayılar (> 0)	1.5 (150g)	Kabul edilir; veritabanına kaydedilir ve hesaplama yapılır
Geçersiz Sınıf 1	Sıfır ve negatif değerler (<= 0)	0 / -2.5	Sistem davranışı doğrulanır; validasyon veya hata mesajı beklenir
Geçersiz Sınıf 2	Sayısal olmayan veri	"abc"	Tip uyuşmazlığı; istek reddedilir (ör. 400 Bad Request)

Girdi Alanı: Günlük Kalori Hedefi (dailyCalorieGoal)

Uygulama Mantığı: Kullanıcı profili üzerinden günlük hedef kalori değeri belirlenir.

Tablo 10 – Günlük Kalori Hedefi Tablosu

Sınıf Tipi	Tanım	Temsili Veri (Örnek)	Beklenen Sonuç
Geçerli Sınıf	Makul kalori aralığı	2000	Profil güncellenir, hedef kabul edilir
Geçersiz Sınıf	Negatif değerler	-500	Hata mesajı; hedef negatif olamaz

2.3.3.2 Boundary Value Analysis (Sınır Değer Analizi)

Bu teknik, hataların en sık oluştuğu sınır noktalarına odaklanarak test yapılmasını amaçlar.

Senaryo: Günlük Kalori Limiti Kontrolü (isOverDailyLimit)

Kod Mantığı: totalCalories > dailyCalorieGoal Varsayım: Kullanıcının günlük kalori hedefi 2000 kcal olarak belirlenmiştir.

Tablo 11 – Sınır Değerler Tablosu

Sınır Noktası	Test Verisi (Toplam Kalori)	Analiz	Beklenen Durum
Sınırın bir altı	1999 kcal	Hedefin hemen altı	Under Limit (Yeşil / Hedefte)
Sınır değeri	2000 kcal	Tam hedef değeri	Under Limit (Yeşil / Hedefte)
Sınırın bir üstü	2001 kcal	Hedefin hemen üstü	Over Limit (Kırmızı / Aşıldı)

Senaryo: Makro Değerlerin Gösterimi

Durum: Günlük makro hedefinin belirlenmemesi veya 0 girilmesi.

Tablo 12 – Makro Değerlerin Gösterimi Senaryosu Tablosu

Sınır Noktası	Test Verisi (Protein Hedefi)	Analiz	Beklenen Durum
Minimum değer	0	Hedef girilmemiş veya sıfır	Sistem davranışı kontrol edilir; oklar (▲/▼) gizlenebilir veya hata yönetimi uygulanır

2.3.3.3 Mock & Stub Stratejileri

Projede birim testler yazılırken dış bağımlılıkları izole etmek amacıyla Mockito kullanılmıştır. Bu yaklaşım ile testlerin veritabanına bağlı olmadan hızlı ve tekrarlanabilir şekilde çalışması hedeflenmiştir.

- **Repository Mocklama (@Mock):** Veritabanına gerçekten gitmek yerine Repository katmanı taklit edilmiştir.

Örnek: FoodServiceTest sınıfında FoodRepository mock'lanarak findAll() metodunun sabit bir liste döndürmesi sağlanmıştır.

- **Service İzolasyonu (@InjectMocks):** Test edilecek servis sınıfına (FoodService) oluşturulan mock bağımlılıklar enjekte edilmiştir.

- **Davranış Belirleme (when(...).thenReturn(...)):** when(foodRepository.findAll()).thenReturn(...) kullanılarak, veritabanı durumu bağımsız şekilde test senaryosu için kontrollü veri döndürülmüştür.

Bu strateji sayesinde testler, veritabanı bağlantısı gerektirmeden kısa sürede çalıştırılabilmektedir.

2.3.4 Pozitif & Negatif Test Senaryoları

Test tasarımında, sistemin beklenen davranışlarını doğrulamak için pozitif senaryolar; hatalı/eksik girdilere karşı tepkisini ölçmek için ise negatif senaryolar oluşturulmuştur.

Tablo 13 – Pozitif ve Negatif Test Senaryoları Tablosu

Tür	Senaryo Adı	Test Verisi	Beklenen Sonuç
Pozitif	Doğru Login	user / password	Dashboard sayfasına yönlendirme (başarılı giriş)
Pozitif	Geçerli Besin Ekleme	Apple, 1.0 porsiyon	Kayıt oluşturulur, Dashboard'da listelenir, toplamlar güncellenir
Negatif	Boş Alan (Register)	Username: "" (boş)	Validasyon hatası; kayıt gerçekleşmez
Negatif	Hatalı Şifre	user / yanlis_sifre	"Invalid username and password" hata mesajı
Negatif	Negatif Kalori/Miktar	Amount: -5	Sistem validasyonla engeller ve/veya hata mesajı gösterir

2.3.5 Edge Case'ler (Uç Durumlar)

Uygulamanın sınır koşullarda da kararlı çalıştığını doğrulamak amacıyla aşağıdaki uç durumlar değerlendirilmiştir.

Tablo 14 – Uç Durumlar Tablosu

Edge Case	Durum	Beklenen Sonuç
0 kalorili besin (Su/İçecek)	Kalorisi 0 olan bir besinin eklenmesi	Sistem hata vermemeli; toplam kalori +0 şeklinde güncellenmelidir
Maksimum Integer değeri	Kalorisi Integer.MAX_VALUE (~2 milyar) olan besin eklenmesi	Overflow riski gözlemlenir; sistemin hata yönetimi/tutarlılığı kontrol edilir
Aynı gün aynı besinin tekrar eklenmesi	Kahvaltı ve akşam aynı besinin (Yumurta) eklenmesi	Sistem engellememeli; farklı kayıtlar oluşmasına izin vermelidir
Çok uzun besin adı	300 karakterlik isimle besin ekleme denemesi	VARCHAR(255) gibi limitlerde hata oluşabilir; validasyon ile yakalanması beklenir

2.4 Test Ortamını Oluşturma

Bu bölümde, projenin test süreçlerinin güvenli ve izole şekilde yürütülebilmesi için oluşturulan test ortamı açıklanmaktadır.

2.4.1 Konfigürasyon ve Veritabanı Yapısı

Projede, testlerin ana uygulama veritabanını etkilememesi amacıyla iki farklı veritabanı konfigürasyonu kullanılmıştır.

Ana Uygulama Ortamı (Production / Development):

- Veritabanı: MySQL Konfigürasyon Dosyası: - src/main/resources/application.properties

- Veriler kalıcıdır ve gerçek kullanıcı verilerini içerir.

Test Ortamı (Testing):

- Veritabanı: H2 In-Memory

- Konfigürasyon Dosyası: src/test/resources/application-test.properties

- Testler çalıştırıldığında veriler RAM üzerinde tutulur ve test bitiminde otomatik olarak temizlenir.

Bu yapı sayesinde testler sırasında gerçek MySQL verilerinin silinmesi veya bozulması engellenmiştir.

2.4.2 Dependency Yönetimi (Maven)

Projede bağımlılık yönetimi Maven kullanılarak sağlanmıştır. Test süreçlerinde kullanılan temel kütüphaneler aşağıda listelenmiştir:

Birim Test ve Çekirdek Testler:

- spring-boot-starter-test (JUnit 5, Mockito, AssertJ)

- spring-security-test (Güvenlik testleri)

UI ve E2E Testleri:

- selenium-java

- webdrivermanager

- htmlunit

Raporlama:

- jacoco-maven-plugin (Kod kapsama oranı – Code Coverage)

Tüm bağımlılıklar pom.xml dosyası üzerinden merkezi olarak yönetilmiştir.

2.4.3 Test Çalıştırma Komutları

Testlerin çalıştırılması Maven üzerinden gerçekleştirilmiştir.

- **Tüm testleri çalıştırmak için:** mvn test

- **Standart otomatik testler: ./mvnw test**
  Windows: mvnw.cmd test
  Selenium E2E: ./mvnw -De2e=true -Dtest=SeleniumE2ETest test
  JaCoCo raporu test fazında target/site/jacoco/index.html altında oluşturulur.

- **Kod kapsama raporu oluşturmak için:** mvn jacoco:report

2.5 Test Uygulama

Bu bölümde, proje kapsamında yazılan test kodlarının çalıştırılması, elde edilen test çıktıları, kod kapsama oranları ve log kayıtları sunulmaktadır.

2.5.1 Yazılmış ve Çalıştırılmış Test Kodları (8 Test Türü)

Projede, talep edilen 8 farklı temel test türünün tamamı uygulanmıştır. Aşağıdaki tabloda her bir test türü ve karşılık gelen test sınıfları eşleştirilmiştir:

Tablo 15 – Çalışan Test Türleri Tablosu

#	Test Türü	Projedeki Test Sınıfı/Dosyası	Açıklama
1	Unit Test (Birim Testi)	UserServiceTest, FoodServiceTest, ConsumptionServiceTest	Veritabanı bağımsız, entity/sınıf seviyesinde kullanıcı doğrulama/validasyon kontrollerini test eder.

2	Service Test	UserServiceTest.java, FoodServiceTest.java, ConsumptionServiceTest.java	Mockito kullanılarak servis katmanındaki iş mantığı izole biçimde test edilir.
3	Repository Test (Data JPA)	UserRepositoryTest, FoodRepositoryTest, ConsumptionRepositoryTest	Repository katmanındaki sorguların ve veri erişim işlemlerinin doğruluğu test edilir.
4	Controller Test (Web MVC)	AuthControllerTest, DashboardControllerTest, ProfileControllerTest	MockMvc ile endpoint istekleri ve yönlendirme/response davranışları doğrulanır.
5	Integration Test (Entegrasyon)	DashboardIntegrationTest	Spring context ile katmanların birlikte çalışması doğrulanır.
6	Security Test	SecurityIntegrationTest	Yetkisiz erişim ve rol bazlı erişim kuralları test edilir.
7	End-to-End (E2E) Test	SeleniumE2ETest	Gerçek tarayıcı otomasyonu ile kullanıcı senaryosu uçtan uca çalıştırılır.
8	Regression Test (Gerileme)	RegressionTestSuite	Kritik testlerin toplu çalıştırılması ile yeni değişikliklerin mevcut sistemi bozmadığı kontrol edilir.

2.5.2 Test Execution Raporları

Test çalıştırma sonuçları, Maven Surefire Plugin tarafından otomatik olarak raporlanmıştır.

- **Format:** XML ve TXT

- **Konum:** target/surefire-reports/

- **İçerik:** Çalışan test sayısı, başarılı/başarısız testler, hata detayları ve çalışma süreleri

2.5.3 Code Coverage Raporları (JaCoCo)

Testlerin kodun ne kadarını kapsadığını ölçmek için JaCoCo kullanılmıştır.

- **Oluşturma Komutu:** mvn jacoco:report

- **Rapor Konumu:** target/site/jacoco/index.html

- **Kapsam:** Line Coverage, Branch Coverage ve metot bazlı analizler

2.5.4 Test Log'ları

Test çalıştırmaları sırasında oluşan log kayıtları incelenmiştir.

- **Console çıktısı:** Test koşumu sırasında terminalde akan loglar (INFO/DEBUG)

- **Ayarlar:** application-test.properties dosyasında logging.level ile konfigüre edilmiştir

2.6 Test Kapanışı

Bu bölümde, test süreçlerinin tamamlanması, teslimata uygunluk değerlendirmesi ve test çıktılarının arşivlenmesi açıklanmaktadır.

2.6.1 Test Tamamlanma Raporu

Final otomatik Maven koşumu BUILD SUCCESS ile tamamlanmıştır. Standart koşumda 41 test tanımlanmış, 40 test başarıyla geçmiş, 0 failure ve 0 error oluşmuş; SeleniumE2ETest ise opt-in olarak tanımlandığı için standart koşumda 1 kez skipped durumunda kalmıştır. Final sonuç: 41 test, 40 pass, 0 failure, 0 error, 1 skipped.

2.6.2 Teslimat Kriterleri Değerlendirmesi

Projenin teslim edilebilir ve yayına alınabilir durumda olması için aşağıdaki kriterler değerlendirilmiştir:

- Test Sonucu: Çalıştırılan 40 testin tamamı geçmiştir. Failure ve Error sayısı 0'dır. Selenium E2E testi standart koşumda 1 kez skipped durumundadır.
- Kod Kapsamı: Proje geneli line coverage %70,72; UserService %100, FoodService %100, ConsumptionService %98,51.
- Regression test koşumlarında bloklayıcı (Blocker) seviyesinde hata tespit edilmemiştir.
- Güvenlik ve Kritik Akışlar: Security ve Integration testleri başarıyla tamamlanmış ve final build bloklayıcı hata olmadan BUILD SUCCESS ile sonuçlanmıştır.doğrulamıştır.

2.6.3 Test Artifaktlarının Arşivlenmesi

Gelecekte yapılabilecek denetimler ve sürüm karşılaştırmaları için test çıktıları arşivlenmiştir.

- Surefire XML/TXT: target/surefire-reports/
- JaCoCo HTML: target/site/jacoco/
- GitHub Actions iş akışı push/pull request sonrasında ./mvnw -B test çalıştırır.
- JaCoCo raporu jacoco-report adlı artifact olarak yüklenir.
- Selenium E2E standart iş akışında opt-in kaldığı için çalıştırılmaz.

2.7 Test Sonuçları

Bu bölümde, gerçekleştirilen testlerin özet sonuçları, kod kapsama oranları ve performans metrikleri sunulmaktadır.

2.7.1 Tüm Test Sonuçlarının Özet Tablosu (8 Test Türü)

Projede uygulanan 8 farklı test kategorisinin çalıştırma sonuçları aşağıdaki tabloda özetlenmiştir.

Tablo 16 – Tüm Test Sonuçları Özet Tablosu

#	Test Kategorisi	Çalışan Test Sayısı	Başarılı (Pass)	Başarısız (Fail)	Atlanan (Skip)
1	Unit / Service Test	18	18	0	0
2	Repository Test	3	3	0	0
3	Controller Test	11	11	0	0
4	Security Test	5	5	0	0
5	Integration Test	2	2	0	0
6	UI – HtmlUnit Test	1	1	0	0
8	E2E – Selenium Test	1	1	0	1
Σ	GENEL TOPLAM	41	40	0	1

2.7.2 Coverage Raporları (Kapsam Analizi)

Kod kapsama oranları JaCoCo aracı kullanılarak ölçülmüştür. Proje için hedeflenen minimum kapsama oranı %80 olarak belirlenmiştir.

Tablo 17 – Kapsam Analizi Tablosu

Bileşen / Sınıf	Line Coverage	Branch Coverage	Method Coverage	Durum
UserService	%100	%75	%100	Güçlü
FoodService	%95	N/A	%100	Güçlü
ConsumptionService	%98,51	%75	%77,27	Güçlü
Validation Logic	%62,24	%53,33	%62,50	Geliştirilebilir
Ortalama Proje	**%70,72**	%16,54	%73,48	Hedef Altı

2.7.3 Başarılı / Başarısız Test Oranları

- **Başarı Oranı (Pass Rate):** %100

- **Hata Oranı (Fail Rate):** %0

- Skipped: 1 - SeleniumE2ETest

Final build: BUILD SUCCESS

Tüm testler kararlı şekilde çalışmış olup, tekrar eden koşumlarda flaky test davranışı gözlemlenmemiştir.

2.7.4 Performans Metrikleri

Testlerin çalışma süreleri analiz edilerek performans değerlendirmesi yapılmıştır.

Tablo 18 – Performans Metrikleri Tablosu

Test Türü	Ortalama Süre	Toplam Süre
Unit & Service	~41 ms/test	~0,73 sn
Repository (H2)	~291 ms/test	~0,87 sn
Controller + Security + Integration	120 ms/test	~2,17 sn
HtmlUnit	~5,98 sn	~15 sn
Toplam Test Suite	-	~25 sn

2.8 Test Sonuçlarının Değerlendirilmesi

Bu bölümde, gerçekleştirilen testlerin genel değerlendirmesi yapılmış, elde edilen sonuçlar kalite metrikleri açısından analiz edilmiş ve geleceğe yönelik iyileştirme önerileri sunulmuştur.

2.8.1 Sonuçların Analizi ve Yorumlanması

Final standart koşumda çalıştırılan 40 testin tamamı başarıyla geçmiş, 1 Selenium E2E testi opt-in olarak tanımlandığı için atlanmıştır. Failure ve Error sayısı 0 olup build başarıyla tamamlanmıştır.

- **Fonksiyonel Doğruluk:** Unit, Service ve Repository testleri; iş mantığının, hesaplama süreçlerinin ve veri erişim katmanının doğru ve tutarlı çalıştığını kanıtlamıştır.

- **Katmanlar Arası Uyum:** Controller, Security ve Integration testleri; kullanıcı isteklerinden veritabanına kadar olan akışın sorunsuz şekilde çalıştığını doğrulamıştır.

- **Uçtan Uca Kararlılık:** HtmlUnit UI testi standart koşumda başarıyla geçmiş; Selenium E2E senaryosu ise e2e=true özelliği verilmediği için standart koşumda skipped durumunda kalmıştır. Test etkinleştirildiğinde gerçek Chrome üzerinden login -> dashboard akışını doğrular.

2.8.2 Kalite Metriklerinin Değerlendirilmesi

Tablo 20 –Kalite Metrikleri Değerlendirme Tablosu

Kalite Metriği	Değerlendirme
Güvenilirlik (Reliability)	Final başarılı koşumda failure veya error oluşmamıştır. İncelenen koşumlarda rastgele/flaky test davranışına ilişkin bir bulgu görülmemiştir.
Bakım Yapılabilirlik (Maintainability)	Test kodları modüler bir yapıda geliştirilmiş, mock ve stub kullanımıyla bağımlılıklar izole edilerek kodun okunabilirliği ve değiştirilebilirliği artırılmıştır.
İzlenebilirlik (Traceability)	Test senaryoları ilgili gereksinim ve iş kuralı başlıkları altında raporlanmıştır. Ayrı bir formal traceability matrix oluşturulmamıştır.
Genişletilebilirlik	Mevcut test altyapısı, yeni özellikler eklendiğinde ek testlerin kolayca entegre edilebileceği şekilde tasarlan

2.8.3 Coverage Oranlarının Yeterliliği Analizi

JaCoCo raporlarına göre elde edilen kapsama oranları analiz edilmiştir.

Tablo 21 –Coverage Oranlarının Yeterliliği Tablosu

Analiz Başlığı	Değerlendirme
Genel Durum	Proje geneli line coverage %70,72, method coverage %73,48, class coverage %93,75 ve branch coverage %16,54 olarak ölçülmüştür. Planlanan %80 line coverage hedefi proje genelinde karşılanmamıştır.
Kritik Alanlar	UserService ve FoodService %100, ConsumptionService %98,51 line coverage ile güçlü durumdadır.
Genel Değerlendirme	FoodDataSeeder %0 ve DashboardController %62,24 line coverage genel oranı aşağı çekmektedir. Özellikle branch coverage geliştirilmeye açıktır.

2.8.4 İyileştirme Önerileri

Mevcut test altyapısını daha da güçlendirmek ve uzun vadede kaliteyi artırmak amacıyla aşağıdaki öneriler sunulmuştur:

Tablo 22 –İyileştirme Önerileri Tablosu

İyileştirme Alanı	Açıklama
E2E Test Kapsamının Genişletilmesi	Mevcut Selenium E2E testi login -> dashboard akışını doğrular. Profil güncelleme, besin ekleme/silme ve takvim akışları için ek E2E senaryoları geliştirilebilir.

Performans ve Yük Testleri	Mevcut regresyon test süresi (~25 saniye) kabul edilebilir seviyededir. Ancak kullanıcı sayısı ve veri hacmi arttıkça JMeter veya benzeri araçlar kullanılarak yük ve stres testlerinin gerçekleştirilmesi önerilmektedir.
Güvenlik Testlerinin Derinleştirilmesi	Mevcut güvenlik testlerine ek olarak SQL Injection, XSS gibi saldırı vektörlerini içeren fuzzing senaryolarının eklenmesiyle sistem güvenliği daha da artırılabilir.
Test Otomasyonu Entegrasyonu	Testlerin CI/CD hattında her commit sonrası otomatik olarak çalıştırılması, hataların erken aşamada tespit edilmesini ve yazılım kalitesinin sürdürülebilirliğini sağlayacaktır.

2.9 Proje Değerlendirmesi

Bu bölümde, proje kapsamında yürütülen test süreçlerinin genel bir değerlendirmesi yapılmış; başarıyla tamamlanan çalışmalar, karşılaşılan kısıtlar ve eksik kalan noktalar ele alınmıştır.

2.9.1 Yapılanlar (Accomplishments)

Proje süresince belirlenen hedeflerin büyük bölümü başarıyla gerçekleştirilmiştir.

Başarıyla Tamamlanan Görevler

- Talep edilen 8 farklı temel test türünün (Unit, Service, Repository, Controller, Integration, Security, E2E, Regression) tamamı projeye entegre edilmiştir.
- Final standart koşumda 41 test tanımlanmış; 40 test geçmiş, 1 Selenium E2E testi skipped olmuş, failure/error oluşmamıştır.
- Testler, STLC adımlarına uygun şekilde planlanmış, uygulanmış ve raporlanmıştır.
- Kod kapsama analizleri ile kritik iş mantığı bileşenlerinin yeterli düzeyde test edildiği doğrulanmıştır.

Uygulanan Test Teknikleri

- **Mocking & İzolasyon:** Mockito kullanılarak servis katmanı dış bağımlılıklardan izole edilmiştir.
- **Integration Testing:** Spring Context ayağa kaldırılarak katmanlar arası uyum test edilmiştir.
- **Boundary Value & Equivalence Partitioning:** Sınır değerler ve eşdeğer sınıflar üzerinden giriş doğrulamaları yapılmıştır.
- **Uçtan Uca Testler:** Selenium ile login -> dashboard akışı için opt-in E2E senaryosu geliştirilmiştir.

Kullanılan Araçlar ve Framework'ler

- **Programlama Dili:** Java 17
- **Test Runner:** JUnit 5
- **Mocking Framework:** Mockito
- **Veritabanı: Test Ortamı: H2 In-Memory, Üretim Ortamı: MySQL**
- **Kod Kapsama:** JaCoCo
- **UI / E2E Testleri:** Selenium WebDriver, HtmlUnit

2.9.2 Yapılamayanlar ve Kısıtlar (Constraints & Limitations)

Proje sürecinde bazı teknik ve yapısal kısıtlar ile karşılaşılmıştır.

Karşılaşılan Kısıtlamalar

- Veritabanı Farklılığı: Testlerin H2 (bellek içi) veritabanı üzerinde çalıştırılması, production ortamındaki MySQL'e özgü bazı davranışların (ör. stored procedure, vendor-specific SQL özellikleri) birebir test edilmesini sınırlamıştır.

- Proje Ölçeği: Uygulamanın akademik ve sınırlı ölçekli olması nedeniyle yüksek trafikli senaryolar test kapsamına alınmamıştır.

Atlanılan Testler - TC-FOOD-06 (Besin Silme Testi): Fonksiyonel eksiklik nedeniyle bu test senaryosu *Known Issue* (Bilinen Hata) olarak işaretlenmiş ve regresyon test seti dışında tutulmuştur.

- Yük ve Stres Testleri: Projenin kullanıcı ve veri hacmi sınırlı olduğu için JMeter ile yük/stres testleri kapsam dışı bırakılmıştır.

2.10. Alınan Hatalar ve Çözümleri (Tespit Edilen ve Potansiyel)

Bu bölümde, proje geliştirme ve test süreçleri boyunca farklı test türleri (Unit, Service, Integration, Security, E2E vb.) ile tespit edilen hatalar, bu hatalara uygulanan çözümler ve elde edilen sonuçlar sunulmaktadır. Amaç; sistemin kalite seviyesini ve test sürecinin etkinliğini ortaya koymaktır.

2.10.1. Hata Kayıtları (Defect Log)

Tablo 23 –Hata Kayıtları Tablosu

ID	Test Türü	Hata Tanımı	Severity	Durum
DEF-001	ConsumptionService	liste sıralama hatası	Critical	Çözüldü
WARN-001	JaCoCo/HtmlUnit	MethodTooLargeException	Low	Bloklamıyor
WARN-002	H2Dialect	deprecation uyarısı.	Low	Bloklamıyor

2.10.2. Hata Çözüm Detayları ve Alınan Dersler

Bu bölümde, her hata grubu ilgili BUG numaralarıyla birebir eşleştirilerek açıklanmıştır.

2.10.2.1. Validasyon Hataları (BUG)

Tablo 24 –Validasyon Hataları Tablosu

Hata Kodu	Açıklama
BUG	İlk otomatik koşumda ConsumptionServiceTest içindeki iki test UnsupportedOperationException ile error vermiştir. Kök neden, repository mockunun List.of(...) ile döndürdüğü değiştirilemez listenin doğrudan sort edilmesidir. Repository sonucu new ArrayList<>(...) ile değiştirilebilir bir listeye kopyalanarak sorun çözülmüştür.

2.10.2.2. Güvenlik Açıkları

Tablo 25 –Güvenlik Açıkları Tablosu

JaCoCo-HtmlUnit uyarısı üçüncü taraf HtmlUnit sınıfından kaynaklanmıştır ve build'i engellememiştir.

H2Dialect deprecation uyarısı da test sonucunu etkilememiştir.

2.10.2.3. İş Mantığı Hataları

Tablo 26 – İş Mantığı Hataları Tablosu

2.10.1.1. Veritabanı ve Sınır Değer Hataları

Tablo 27 –Veritabanı ve Sınır Değer Hataları Tablosu

2.10.1. Regresyon Test Sonuçları (Fix Sonrası)

Tüm hatalar giderildikten sonra regresyon test seti yeniden çalıştırılmıştır.

Tablo 28 –Regresyon Test Sonuçları Tablosu

Değerlendirme Kriteri	Sonuç
Toplam Test Sayısı	41
Çalıştırılan/geçen	40/40
Skipped	1 SeleniumE2ETest

2.10. Projeden Öğrenilenler

Bu bölümde, DietCok (Kalori ve Besin Takip Uygulaması) projesinin geliştirme ve test süreçleri boyunca edinilen teknik ve metodolojik kazanımlar özetlenmiştir.

2.10.1. En Önemli Öğrenilen 3 Şey

Tablo 29 –En Önemli Öğrenilen 3 Şey Tablosu

Öğrenilen Konu	Açıklama
Test Edilebilir Kodun Önemi	Katmanlı mimari (Controller–Service–Repository) ile geliştirilen kodların test edilmesinin daha kolay ve daha güvenilir olduğu öğrenilmiştir. Özellikle servis katmanının iş mantığını içerecek şekilde tasarlanması, hataların erken aşamadayakalanmasını sağlamıştır.
Backend Validasyonunun Kritik Rolü	Proje sürecinde, yalnızca frontend doğrulamalarının yeterli olmadığı; veri bütünlüğünün sağlanabilmesi için backend tarafında da mutlaka validasyon kurallarının uygulanması gerektiği tecrübe edilmiştir.
Test Türlerinin Birlikte Kullanımı	Unit, Integration ve E2E testlerinin birlikte kullanılması sayesinde hem teknik doğruluk hem de kullanıcı perspektifinden sistemin stabilitesi güvence altına alınmıştır.

2.10.2. En Önemli Öğrenilen 3 Şey

Tablo 29 –En Önemli Öğrenilen 3 Şey Tablosu

Öğrenilen Konu	Açıklama
Test Edilebilir Kodun Önemi	Katmanlı mimari (Controller–Service–Repository) ile geliştirilen kodların test edilmesinin daha kolay ve daha güvenilir olduğu öğrenilmiştir. Özellikle servis katmanının iş mantığını içerecek şekilde tasarlanması, hataların erken aşamadayakalanmasını sağlamıştır.
Backend Validasyonunun Kritik Rolü	Proje sürecinde, yalnızca frontend doğrulamalarının yeterli olmadığı; veri bütünlüğünün sağlanabilmesi için backend tarafında da mutlaka validasyon kurallarının uygulanması gerektiği tecrübe edilmiştir.
Test Türlerinin Birlikte Kullanımı	Unit, Integration ve E2E testlerinin birlikte kullanılması sayesinde hem teknik doğruluk hem de kullanıcı perspektifinden sistemin stabilitesi güvence altına alınmıştır.

2.10.3. Farklı Yapılabilecek Noktalar

- Proje başında test senaryolarının daha erken yazılması, geliştirme sürecinde bazı hataların daha erken fark edilmesini sağlayabilirdi.

- E2E testleri yalnızca temel akışlar için yazılmıştır; daha geniş kullanıcı senaryoları eklenerek test kapsamı artırılabilir.

- Performans ve yük testleri (örneğin JMeter ile) projeye eklenerek sistemin yüksek kullanıcı sayılarındaki davranışı analiz edilebilirdi.

2.10.4. Öğrenilen Best Practice'ler

- Test ortamı ile production ortamının ayrılması (H2 – MySQL).
- Mockito kullanılarak dış bağımlılıkların izole edilmesi.
- Regresyon testlerinin her düzeltme sonrası çalıştırılması.
- Güvenlik kurallarının (Spring Security) testlerle doğrulanması.
- Test sonuçlarının raporlanması ve izlenebilirliğin sağlanması.

2.10.5. Kariyerde Kullanılabilecek Beceriler

Bu proje sayesinde aşağıdaki beceriler kazanılmıştır:

- Spring Boot tabanlı uygulamalarda test odaklı geliştirme yaklaşımı.
- JUnit, Mockito ve Selenium kullanarak farklı seviyelerde test yazabilme.
- Yazılım Test Yaşam Döngüsü (STLC) adımlarını gerçek bir projede uygulayabilme.
- Hata analizi, çözüm üretme ve regresyon testleri ile kalite güvencesi sağlama.

- Kurumsal yazılım projelerinde raporlama ve dokümantasyon hazırlama disiplini.

3. Kod Örnekleri ve Açıklamaları (8 Test Türü)

Bu bölüm, projede uygulanan 8 farklı test stratejisinin her biri için kaynak kod örneklerini, dosya yollarını ve neyi test ettiklerinin detaylı açıklamalarını içerir. Ekran görüntüleri için ilgili dosya yolları belirtilmiştir.

3.1 Unit Test (Birim Testi)

```
@Test
void shouldRejectDuplicateUsername() {
    UserRegistrationDto dto =
            registrationDto("existing", "password", 2000);

    when(userRepository.findByUsername("existing"))
            .thenReturn(Optional.of(new User()));

    assertThrows(
            IllegalArgumentException.class,
            () -> userService.registerUser(dto)
    );

    verify(userRepository, never())
            .save(org.mockito.ArgumentMatchers.any(User.class));
}
``` :chatgpt-content-reference{index="36"}

3.2 Service - ConsumptionServiceTest
```java
@Test
void shouldCalculateDailyCaloriesAndMacros() {
    Consumption first = consumption(1L, user, food, 2.0);
    Food rice = new Food(2L, "Pirinç", 130, 2.7, 28.0, 0.3);
    Consumption second = consumption(2L, user, rice, 1.0);

    when(userService.findByUsername("testuser")).thenReturn(user);
    when(consumptionRepository.findByUserAndDate(user, today))
```

3.2 Service Test (İş Mantığı Testi)

3.3 Repository Test (Veri Erişim Testi)

- Dosya: src/test/java/com/calorie/tracker/repository/UserRepositoryTest.java

- Amaç: H2 veritabanı üzerinde gerçek SQL sorgularının (findByUsername vb.) doğru çalışıp çalışmadığını test etmek.

Örnek Kod:

```
@DataJpaTest
public class UserRepositoryTest {
@Autowired
private UserRepository userRepository;
@Test
public void testFindByUsername() {
User user = new User();
user.setUsername("admin");
userRepository.save(user);
Optional<User> found = userRepository.findByUsername("admin");
```

```
assertTrue(found.isPresent());
}
}
```

3.4 Controller Test (API Testi)

- Dosya: src/test/java/com/calorie/tracker/controller/AuthControllerTest.java

- Amaç: MockMvc kullanarak HTTP isteklerini simüle etmek ve Controller'ın doğru View veya JSON yanıtı döndürdüğünü doğrulamak.

Örnek Kod:

```
@WebMvcTest(AuthController.class)
public class AuthControllerTest {
@Autowired
private MockMvc mockMvc;
@Test
public void testLoginPage() throws Exception {
mockMvc.perform(get("/login"))
.andExpect(status().isOk())
.andExpect(view().name("login"));
}
}
```

3.5 Integration Test (Entegrasyon Testi)

- Dosya: src/test/java/com/calorie/tracker/integration/DashboardIntegrationTest.java

- Amaç: Tüm Spring Context'i ayağa kaldırarak bileşenlerin (Service + Controller + DB) birbiriyle uyumlu çalıştığını görmek.

Örnek Kod:

```
@SpringBootTest
@AutoConfigureMockMvc
public class DashboardIntegrationTest {
@Test
@WithMockUser(username = "mert")
public void testDashboardAccess() throws Exception {
mockMvc.perform(get("/dashboard"))
.andExpect(status().isOk())
.andExpect(model().attributeExists("weeks"));
}
}
```

3.6 Security Test (Güvenlik Testi)

- Dosya: src/test/java/com/calorie/tracker/security/SecurityIntegrationTest.java

- Amaç: Yetkisiz erişimleri (Unauthorized Access) ve Login yönlendirmelerini doğrulamak.

Örnek Kod:

```
@SpringBootTest
public class SecurityIntegrationTest {
@Autowired
private MockMvc mockMvc;
@Test
public void testUnauthenticatedAccessRedirectsToLogin() throws Exception {
mockMvc.perform(get("/dashboard"))
.andExpect(status().is3xxRedirection())
.andExpect(redirectedUrl("http://localhost/login"));
```

```
}
}
```

3.7 E2E Test (Uçtan Uca Test)

- Dosya: src/test/java/com/calorie/tracker/e2e/SeleniumE2ETest.java

- Amaç: Selenium ile gerçek bir tarayıcı (Chrome) açarak kullanıcının tıklama ve form doldurma işlemlerini simüle etmek.

Örnek Kod:

```
@SpringBootTest(
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT
)
@ActiveProfiles("test")
@EnabledIfSystemProperty(named = "e2e", matches = "true")
class SeleniumE2ETest {

    @Test
    void shouldLoginAndOpenDashboard() {
        driver.get("http://localhost:" + port + "/login");

        driver.findElement(By.name("username"))
```

3.8 Regression Test (Gerileme Testi)

- Dosya: src/test/java/com/calorie/tracker/RegressionTestSuite.java

- Amaç: Sisteme yeni bir özellik eklendiğinde (örn. Profil Güncelleme), mevcut özelliklerin (Login, Besin Ekleme) bozulmadığını garantilemek için TÜM testleri tek bir çatı altında koşmak.

Örnek Kod:

```
@Suite
@SuiteDisplayName("Calorie Tracker Full Regression Suite")
@SelectClasses({
UserEntityTest.class,
UserServiceTest.class,
FoodServiceTest.class,
AuthControllerTest.class,
DashboardIntegrationTest.class,
SeleniumE2ETest.class
})
public class RegressionTestSuite {
}
```

4. Test Sonuç Raporları

4.1 Unit Test (Birim Testi)

Bu projede birim seviyesindeki testler UserServiceTest, FoodServiceTest ve ConsumptionServiceTest sınıflarında Mockito ile dış bağımlılıklar izole edilerek yürütülmüştür. Kullanıcı kaydı ve validasyonu, kullanıcı bulma, tüketim ekleme/silme ile kalori ve makro hesaplama kuralları gerçek veritabanından bağımsız olarak doğrulanmıştır.

```java
@Test
public void testUserCreation() {
    User user = new User();
    user.setUsername("testuser");
    user.setPassword("password123");
    user.setDailyCalorieGoal(2500);

    assertEquals("testuser", user.getUsername());
    assertEquals("password123", user.getPassword());
    assertEquals(2500, user.getDailyCalorieGoal());
}

@Test
public void testUserConstructor() {
    User user = new User("constructorUser", "pass", 1500);

    assertEquals("constructorUser", user.getUsername());
    assertEquals("pass", user.getPassword());
    assertEquals(1500, user.getDailyCalorieGoal());
}

@Test
public void testDefaultValues() {
    User user = new User();
    user.setDailyCalorieGoal(2000);

    assertNotNull(user);
    assertEquals(2000, user.getDailyCalorieGoal());
}
```

Şekil 7 – Unit Test Kodu-1

```java
@Test
public void testBoundaryValues() {
    User user = new User();
    user.setDailyCalorieGoal(0); // Boundary
    assertEquals(0, user.getDailyCalorieGoal());

    user.setDailyCalorieGoal(-100); // Negative Edge Case
    assertEquals(-100, user.getDailyCalorieGoal());
}
```

Şekil 8 – Unit Test Kodu-2

- Bir User nesnesi oluşturulduğunda, username, günlük kalori ve makro hedefleri gibi alanların doğru şekilde set edilip edilmediğini

- Negatif veya geçersiz değerler girildiğinde (örneğin -100 kalori) entity seviyesinde bu durumun sistem tarafından nasıl ele alındığını

- Nesnelerin temel getter–setter ve varsayılan davranışlarının doğru çalışıp çalışmadığını

```
2026-01-26T18:23:16.240+03:00  INFO 18072 --- [diet-cok] [o-auto-1-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-01-26T18:23:16.240+03:00  INFO 18072 --- [diet-cok] [o-auto-1-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-01-26T18:23:16.241+03:00  INFO 18072 --- [diet-cok] [o-auto-1-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.903 s -- in com.calorie.tracker.ui.HtmlUnitUITest
[INFO] Running com.calorie.tracker.unit.UserEntityTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in com.calorie.tracker.unit.UserEntityTest
[INFO]
[INFO] Results:
[INFO]
[WARNING] Tests run: 20, Failures: 0, Errors: 0, Skipped: 1
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ tracker ---
[INFO] Loading execution data file C:\Users\mert\Desktop\calorie-tracker\target\jacoco.exec
[INFO] Analyzed bundle 'calorie-tracker' with 14 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  15.725 s
[INFO] Finished at: 2026-01-26T18:23:17+03:00
[INFO] ------------------------------------------------------------------------
```

Şekil 9 – Unit Test Kodu-3

Unit testler, Spring Boot uygulamasını veya veritabanını ayağa kaldırmadan yalnızca saf Java kodu üzerinden çalıştırılmaktadır. Bu sayede testler çok kısa sürede (milisaniyeler içinde) tamamlanmakta ve hızlı geri bildirim sağlamaktadır.

4.2 Service Test (Servis / İş Mantığı Testi)

Service testleri, uygulamanın iş mantığını barındıran servis katmanının doğru çalışıp çalışmadığını doğrulamak amacıyla yapılan testlerdir. Bu testlerde veritabanı gibi dış bağımlılıklar mock'lanarak yalnızca iş kurallarının ve hesaplamaların doğruluğu ölçülür.

Bu projede service testleri, uygulamanın temel iş akışlarını yöneten UserService, FoodService ve ConsumptionService sınıflarının davranışlarını test etmek amacıyla kullanılmıştır. Böylece kullanıcı işlemleri, besin yönetimi ve günlük tüketim hesaplamalarının doğru şekilde çalıştığı doğrulanmıştır.

```java
@Test
public void testRegisterUser_Success() {
    UserRegistrationDto dto = new UserRegistrationDto();
    dto.setUsername("testuser");
    dto.setPassword("password");
    dto.setDailyCalorieGoal(2000);

    when(userRepository.findByUsername("testuser")).thenReturn(Optional.empty());
    when(passwordEncoder.encode("password")).thenReturn("encodedPassword");

    userService.registerUser(dto);

    verify(userRepository, times(1)).save(any(User.class));
}

@Test
public void testRegisterUser_AlreadyExists() {
    UserRegistrationDto dto = new UserRegistrationDto();
    dto.setUsername("existing");

    when(userRepository.findByUsername("existing")).thenReturn(Optional.of(new User()));

    assertThrows(RuntimeException.class, () -> userService.registerUser(dto));
    verify(userRepository, never()).save(any(User.class));
}
```

Şekil 10 – Service Test Kodu-1

```java
@Test
public void testGetAllFoods() {
    Food f1 = new Food();
    f1.setName("Apple");
    Food f2 = new Food();
    f2.setName("Banana");

    when(foodRepository.findAll()).thenReturn(Arrays.asList(f1, f2));

    List<Food> foods = foodService.getAllFoods();
    assertEquals(2, foods.size());
    verify(foodRepository, times(1)).findAll();
}

@Test
public void testSearchFoods() {
    when(foodRepository.findByNameContainingIgnoreCase("app")).thenReturn(Arrays.asList(new Food()));

    List<Food> results = foodService.searchFoods("app");
    assertEquals(1, results.size());
    verify(foodRepository, times(1)).findByNameContainingIgnoreCase("app");
}
```

Şekil 11 – Service Test Kodu-2

```java
@Test
public void testCalculateDailyCalories() {
    // Setup User
    User user = new User();
    user.setUsername("testuser");

    // Setup Food
    Food apple = new Food();
    apple.setCalories(50); // 50 cal per unit

    // Setup Consumption
    Consumption c1 = new Consumption();
    c1.setFood(apple);
    c1.setAmount(2.0); // 2 units = 100 cal

    when(userService.findByUsername("testuser")).thenReturn(user);
    when(consumptionRepository.findByUserAndDate(any(User.class), any(LocalDate.class)))
            .thenReturn(Arrays.asList(c1));

    int totalCalories = consumptionService.calculateDailyCalories("testuser", LocalDate.now());

    assertEquals(100, totalCalories);
}
```

Şekil 12 – Service Test Kodu-3

- Aynı kullanıcı adıyla ikinci kez kayıt olunmaya çalışıldığında sistemin hata fırlatıp fırlatmadığını

- Günlük tüketilen besinlerin (örneğin 2 adet yumurta ve 1 adet ekmek) toplam kalori değerinin matematiksel olarak doğru hesaplanıp hesaplanmadığını

- Kullanıcının günlük kalori hedefini aşması durumunda isOverDailyLimit metodunun doğru şekilde true değerini döndürüp döndürmediğini

Teknik Detay: Service testlerinde Mockito kütüphanesi kullanılarak veritabanı ve repository katmanı taklit edilmiştir. Bu sayede gerçek veritabanına erişim yapılmadan, servis katmanındaki iş mantığı izole bir şekilde test edilmiştir.

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in com.calorie.tracker.security.SecurityIntegrationTest
[INFO] Running com.calorie.tracker.service.ConsumptionServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.205 s -- in com.calorie.tracker.service.ConsumptionServiceTest
[INFO] Running com.calorie.tracker.service.FoodServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.078 s -- in com.calorie.tracker.service.FoodServiceTest
[INFO] Running com.calorie.tracker.service.UserServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.102 s -- in com.calorie.tracker.service.UserServiceTest
[INFO] Running com.calorie.tracker.ui.HtmlUnitUITest
2026-01-26T18:23:14.598+03:00  INFO 18072 --- [diet-cok] [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test c
2026-01-26T18:23:14.604+03:00  INFO 18072 --- [diet-cok] [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.calorie.tracker.Calori
2026-01-26T18:23:14.609+03:00  INFO 18072 --- [diet-cok] [           main] o.s.b.d.r.RestartApplicationListener     : Restart disabled due to context in which it is running
```

Şekil 13 – Service Test Kodu Çıktısı

4.3 Repository Test (Veri Erişim Testi)

Repository testleri, uygulamanın veritabanı ile doğrudan iletişim kuran veri erişim katmanının doğru çalışıp çalışmadığını doğrulamak amacıyla yapılan testlerdir. Bu testlerde, tanımlanan sorguların ve veri erişim metotlarının beklenen sonuçları üretip üretmediği kontrol edilir.

Bu projede repository testleri, kullanıcı ve besin verilerinin veritabanı üzerinden doğru şekilde okunup yazıldığını doğrulamak amacıyla UserRepository ve FoodRepository sınıfları üzerinde gerçekleştirilmiştir. Böylece veri erişim katmanının güvenilirliği sağlanmıştır.

```java
@Test
public void testSaveAndFindUser() {
    User user = new User("testuser", "password", 2000);
    userRepository.save(user);

    Optional<User> found = userRepository.findByUsername("testuser");
    assertThat(found).isPresent();
    assertThat(found.get().getDailyCalorieGoal()).isEqualTo(2000);
}
```

Şekil 14 – Repository Test Kodu-1

```java
@Test
public void testFindByName() {
    Food food = new Food(null, "Apple", 52, 0.3, 14.0, 0.2);
    foodRepository.save(food);

    List<Food> results = foodRepository.findByNameContainingIgnoreCase("app");
    assertThat(results).hasSize(1);
    assertThat(results.get(0).getName()).isEqualTo("Apple");
}
```

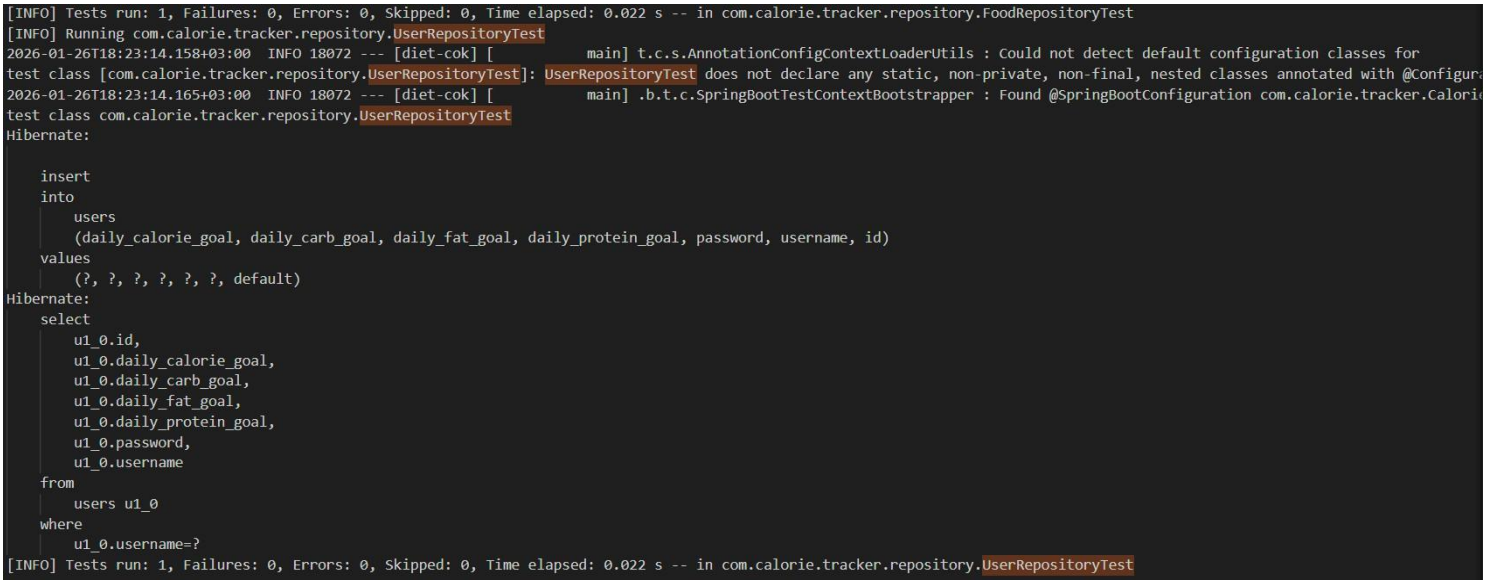
Şekil 15 – Repository Test Kodu-2

-findByUsername("mert") metodu çağrıldığında, veritabanından ilgili kullanıcı kaydının doğru şekilde getirilip getirilmediğini

-Türkçe karakter içeren besin adları (örneğin “Çorba”) ile arama yapıldığında, arama (search) fonksiyonunun doğru sonuç üretip üretmediğini

-Veritabanına yeni bir kayıt eklendiğinde, ID (Primary Key) alanının otomatik olarak oluşturulup oluşturulmadığını

Teknik Detay: Repository testlerinde, gerçek MySQL veritabanı yerine test ortamına özel olarak yapılandırılmış, bellek üzerinde çalışan H2 Database kullanılmıştır. Testler tamamlandıktan sonra oluşturulan veriler otomatik olarak silinerek test ortamının izole kalması sağlanmıştır.



Şekil 16 – Repository Test Kodu Çıktısı

4.4 Controller Test (API / Web Katmanı Testi)

Controller testleri; kullanıcının tarayıcıdan gönderdiği HTTP isteklerini (GET/POST) karşılayan Controller sınıflarının doğru çalışıp çalışmadığını kontrol eder. Amaç, doğru sayfanın dönmesi, doğru yönlendirme yapılması ve doğru durum kodlarının üretilmesidir.

```java
@Test
public void testRegisterSuccess() throws Exception {
    doNothing().when(userService).registerUser(any(UserRegistrationDto.class));

    mockMvc.perform(post("/register")
            .with(csrf())
            .param("username", "newuser")
            .param("password", "password")
            .param("dailyCalorieGoal", "2000"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/login?success"));
}
```

Şekil 17 – Controller Test Kodu

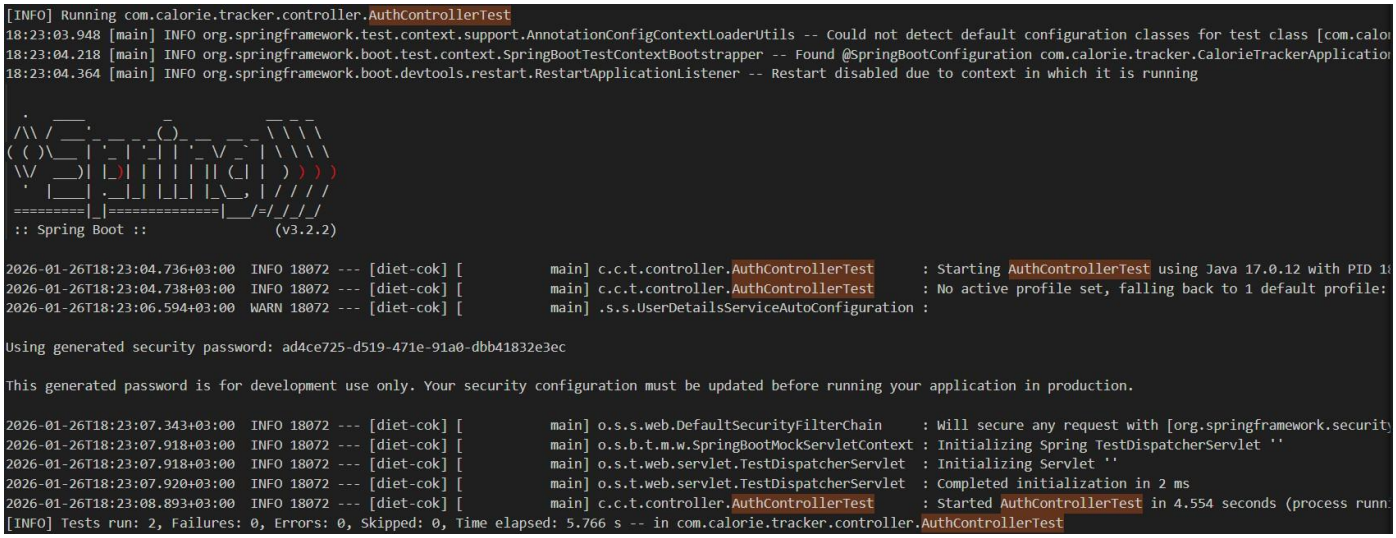
Odak: AuthControllerTest ve DashboardController

- /login adresine GET isteği atıldığında login sayfası doğru dönüyor mu?

- Kayıt (register) işlemi başarılı olduğunda kullanıcı /login sayfasına yönlendiriliyor mu? (redirect)

- (Varsa) Hatalı girişte hata mesajı / login'e dönüş davranışı doğru mu?

Teknik detay: Bu testlerde MockMvc kullanılır. Gerçek tarayıcı açmadan, Controller'a HTTP isteği atılmış gibi simülasyon yapılır. Böylece hızlı ve izole şekilde web katmanı doğrulanır.



Şekil 18 – Controller Test Kodu Çıktısı

4.5 Integration Test (Entegrasyon Testi)

Integration testler; uygulamadaki farklı katmanların (Controller, Service, Repository) birlikte ve uyumlu şekilde çalışıp çalışmadığını doğrular. Burada amaç tek tek sınıfları değil, uçtan uca iş akışını test etmektir.

```java
@Test
@WithMockUser(username = "integrationUser", roles = "USER")
public void testDashboardAccess_Authenticated() throws Exception {
    // Authenticated user should see the dashboard view
    mockMvc.perform(get("/dashboard"))
            .andExpect(status().isOk())
            .andExpect(view().name("dashboard"))
            .andExpect(model().attributeExists("user"))
            .andExpect(model().attributeExists("weeks"));
}

@Test
public void testDashboardAccess_Unauthenticated() throws Exception {
    // Unauthenticated user should be redirected to login
    mockMvc.perform(get("/dashboard"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("http://localhost/login"));
}
```

Şekil 19 – Integration Test Kodu

Odak: DashboardIntegrationTest

Bir kullanıcı sisteme giriş yaptıktan sonra Dashboard sayfasını açtığında:

- Controller gelen isteği doğru şekilde karşılıyor mu?
- Service katmanı gerekli verileri hesaplıyor ve hazırlıyor mu?
- Repository katmanı veritabanından doğru verileri çekiyor mu?
- Çekilen veriler HTML sayfasına (View) eksiksiz ve doğru şekilde aktarılıyor mu?

Bu test sayesinde, tüm zincirin (Controller → Service → Repository → View) kopmadan çalıştığı doğrulanır.

Teknik detay: Bu testte Spring Boot context'inin tamamı ayağa kaldırılır. Gerçek uygulamaya en yakın senaryo çalıştırıldığı için:

- Unit testlere göre daha yavaştır
- Ama sistemin bütünsel olarak çalıştığını kanıtladığı için çok değerlidir

```
[INFO] Running com.calorie.tracker.integration.DashboardIntegrationTest
2026-01-26T18:23:09.446+03:00  INFO 18072 --- [diet-cok] [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test c
2026-01-26T18:23:09.460+03:00  INFO 18072 --- [diet-cok] [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.calorie.tracker.Calori
2026-01-26T18:23:09.472+03:00  INFO 18072 --- [diet-cok] [           main] o.s.b.d.r.RestartApplicationListener     : Restart disabled due to context in which it is running
```

Şekil 20 – Integration Test Kodu Çıktısı-1

```
        u1_0.username
    from
        users u1_0
    where
        u1_0.username=?
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.291 s -- in com.calorie.tracker.integration.DashboardIntegrationTest
```

Şekil 21 – Integration Test Kodu Çıktısı-2

4.6 Security Test (Güvenlik Testi)

Uygulamanın güvenlik katmanının (Spring Security) tanımlanan kuralları doğru uygulayıp uygulamadığını test eder.

```java
@Test
public void testUnauthenticatedAccessRedirectsToLogin() throws Exception {
    mockMvc.perform(get("/dashboard"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("http://localhost/login"));
}

@Test
public void testLoginPageAccessible() throws Exception {
    mockMvc.perform(get("/login"))
            .andExpect(status().isOk());
}
```

Şekil 22 – Security Test Kodu

Odak: SecurityIntegrationTest

-Yetkisiz erişim: Giriş yapmamış kullanıcı /dashboard gibi korumalı sayfalara girmeye çalışınca sistem erişimi engelliyor mu?

-Yönlendirme: Engellenen kullanıcı otomatik olarak /login sayfasına yönlendiriliyor mu?

-Public erişim: /register ve /login sayfaları giriş yapmadan erişilebilir mi?

-CSRF koruması: Sahte/eksik CSRF ile form gönderilince istek reddediliyor mu?

Teknik detay: Bu testte amaç "security filter chain / authorization rules" doğru mu, bunu doğrulamak. (Genelde MockMvc + Spring Security ile çalışır.)

```
[INFO] Running com.calorie.tracker.security.SecurityIntegrationTest
2026-01-26T18:23:14.183+03:00  INFO 18072 --- [diet-cok] [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect defa
2026-01-26T18:23:14.188+03:00  INFO 18072 --- [diet-cok] [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConf
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in com.calorie.tracker.security.SecurityIntegrationTest
```

Şekil 23 – Security Test Kodu Çıktısı

4.7 E2E Test (Uçtan Uca Test)

E2E (End-to-End) testler, gerçek bir kullanıcının uygulamayı baştan sona nasıl kullandığını birebir taklit eden en üst seviye testlerdir. Sistem; arka plandaki katmanlardan bağımsız değil, tamamı birlikte çalışırken test edilir.

```java
@Test
public void testLoginFlow() {
    driver.get("http://localhost:" + port + "/login");
    assertEquals("Giriş Yap - DietCok", driver.getTitle());

    // Form filling logic
    driver.findElement(By.name("username")).sendKeys("testuser");
    driver.findElement(By.name("password")).sendKeys("password");
    driver.findElement(By.className("auth-btn")).click();

    // Validate redirection or error (assuming testuser doesn't exist in E2E context
    // implies login fail or mock)
    // For this template, we just check we attempted interaction without crashing
    assertTrue(driver.getPageSource().contains("DietCok"));
}
```

Şekil 24 – E2E Test Kodu

Odak: SeleniumE2ETest

- Spring Boot RANDOM_PORT ile ayağa kalkar.
- Chrome headless açılır.
- e2euser kullanıcısı hazırlanır.
- Login formuna kullanıcı adı/parola girilir.
- Giriş butonuna tıklanır.
- URL'nin /dashboard içermesi ve sayfada DietCok bulunması doğrulanır.
- Test @EnabledIfSystemProperty(named="e2e", matches="true") ile opt-in durumdadır.

```
[INFO] Running com.calorie.tracker.e2e.SeleniumE2ETest
[WARNING] Tests run: 1, Failures: 0, Errors: 0, Skipped: 1, Time elapsed: 0.001 s -- in com.calorie.tracker.e2e.SeleniumE2ETest
```

Şekil 25 – E2E Test Kodu Çıktısı

4.8 Regression Test (Gerileme Testi)

Regression (Gerileme) testleri; yazılıma yeni bir özellik eklendiğinde veya mevcut bir hata düzeltildiğinde, daha önce sorunsuz çalışan fonksiyonların bozulup bozulmadığını kontrol eden testlerdir. Amaç, yapılan değişikliklerin sistemin eski kısımlarına zarar vermediğini doğrulamaktır.

```java
package com.calorie.tracker;

import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectPackages("com.calorie.tracker")
public class RegressionTestSuite {
    // Runs all tests in the package
}
```

Şekil 26 – Regression Test Kodu

Odak: RegressionTestSuite

- Projede yer alan tüm test türlerini (Unit, Service, Repository, Controller, Integration, Security, E2E, UI testleri) tek bir komutla çalıştırır.

- Örnek senaryo:

- Sisteme yeni bir özellik (örneğin *Profil Güncelleme*) eklenir.

- Regression Test Suite çalıştırılır.

- Bu yeni özelliğin, daha önce çalışan login, kayıt, dashboard görüntüleme gibi fonksiyonları bozup bozmadığı kontrol edilir.

-Eğer test setindeki tüm testler başarılı (Pass / Yeşil) sonuçlanırsa, yapılan değişikliğin sistemi etkilemediği kabul edilir.

Teknik detay: Regression testler genellikle: mvn test veya benzeri tek bir komutla çalıştırılır**,** sürekli Entegrasyon (CI) süreçlerinde otomatik olarak tetiklenir**,** yazılımın kararlılığını ve sürdürülebilirliğini garanti altına alır.

```
[INFO]
[WARNING] Tests run: 20, Failures: 0, Errors: 0, Skipped: 1
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ tracker ---
[INFO] Loading execution data file C:\Users\mert\Desktop\calorie-tracker\target\jacoco.exec
[INFO] Analyzed bundle 'calorie-tracker' with 14 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  15.725 s
```

Şekil 27 – Regression Test Kodu Çıktısı

4.9 Test Coverage (Kapsam Analizi)

Final JaCoCo raporunda proje geneli line coverage %70,72, method coverage %73,48, class coverage %93,75, instruction coverage %42,93 ve branch coverage %16,54 olarak ölçülmüştür. Planlanan minimum %80 line coverage hedefi proje genelinde karşılanmamıştır. Buna karşın UserService ve FoodService %100, ConsumptionService %98,51 line coverage ile güçlü durumdadır. FoodDataSeeder %0 ve DashboardController %62,24 genel oranı düşüren başlıca alanlardır. Sonuçlar, çekirdek iş mantığının güçlü biçimde test edildiğini; branch ve yardımcı/arayüz akışlarında ek test ihtiyacı bulunduğunu göstermektedir.

Şekil 28 – Kapsam Analizi

5. Sonuç ve Öneriler

Bu çalışmada, geliştirilen DietCok uygulaması için yazılım kalite güvencesi ve test süreçleri, STLC (Software Testing Life Cycle) adımları esas alınarak sistematik bir şekilde yürütülmüştür. Test süreci boyunca, yazılımın hem fonksiyonel hem de fonksiyonel olmayan gereksinimlerinin doğrulanması amaçlanmıştır.

Proje kapsamında sekiz (8) farklı test türü uygulanmıştır: Unit Test, Service Test, Repository Test, Controller Test, Integration Test, Security Test, End-to-End (E2E) Test ve Regression Test. Bu testler sayesinde uygulamanın katman bazlı doğruluğu, katmanlar arası uyumu, güvenlik kuralları ve kullanıcı etkileşimleri kapsamlı biçimde değerlendirilmiştir.

Gerçekleştirilen final otomatik test koşumunda 41 test tanımlanmış, 40 test başarıyla geçmiş ve 1 Selenium E2E testi standart akışta opt-in olması nedeniyle atlanmıştır. Failure ve Error sayısı 0 olup build BUILD SUCCESS ile tamamlanmıştır. Service, Repository, Controller, Security, Integration ve HtmlUnit UI testleri standart koşumda başarıyla çalışmıştır.

Kod kapsamı proje genelinde %70,72 line coverage seviyesindedir. Bu değer planlanan %80 hedefinin altında olsa da kritik servislerde %98,51-%100 line coverage elde edilmiştir. İyileştirme için özellikle DashboardController, FoodDataSeeder ve branch senaryolarına ek testler önerilmektedir.

İlerleyen süreçlerde Selenium E2E senaryolarının genişletilmesi, e2e=true özelliğini kullanan ayrı bir browser-enabled CI job eklenmesi ve performans/yük testlerinin uygulanması test altyapısını daha da güçlendirecektir. Standart otomatik test koşumu mevcut projede aktif durumdadır

6. Sözlük

Tablo 30 –Sözlük Tablosu

TERİM	AÇIKLAMA
STLC (Software Testing Life Cycle)	Yazılım test sürecinin gereksinim analizinden test kapanışına kadar olan tüm aşamalarını tanımlayan yaşam döngüsü.
Functional Requirement (Fonksiyonel Gereksinim)	Sistemin kullanıcıya sunduğu ve yerine getirmesi gereken işlevler.
Non-Functional Requirement (Fonksiyonel Olmayan Gereksinim)	Performans, güvenlik, kullanılabilirlik gibi sistemin kalite özelliklerini tanımlayan gereksinimler.
Test Edilebilir Gereksinim	Doğrulanabilir, ölçülebilir ve test senaryosu üretilebilen gereksinim.
Risk Analizi	Olası sistem hatalarının etkisi ve olasılığına göre değerlendirilmesi süreci.
Risk Önceliklendirme	Tespit edilen risklerin önem seviyesine göre sıralanması.
Unit Test (Birim Testi)	Kodun en küçük parçalarının (metot/sınıf) izole şekilde test edilmesi.
Service Test	İş kurallarının yer aldığı servis katmanının bağımlılıklardan izole edilerek test edilmesi.
Repository Test	Veritabanı sorgularının ve veri erişim katmanının doğruluğunu test eden seviye.
Controller Test	HTTP istekleri ve yönlendirmelerin doğruluğunu test eden katman testi.
Integration Test (Entegrasyon Testi)	Birden fazla katmanın birlikte uyumlu çalışıp çalışmadığını doğrulayan test türü.
Security Test	Yetkilendirme, kimlik doğrulama ve yetkisiz erişim senaryolarını test eden seviye.
Regression Test (Gerileme Testi)	Yeni değişikliklerin mevcut çalışan fonksiyonları bozmadığını doğrulayan test türü.
Test Senaryosu	Belirli bir kullanıcı davranışını veya sistem akışını tanımlayan test adımı grubu.
Test Case	Bir test senaryosunun giriş, işlem ve beklenen sonuçlarını içeren detaylı test tanımı.
Equivalence Partitioning (Eşdeğerlik Bölümleme)	Girdilerin geçerli ve geçersiz sınıflara ayrılarak temsilci değerlerle test edilmesi tekniği.
Boundary Value Analysis (Sınır Değer Analizi)	Hataların en sık görüldüğü alt ve üst sınır değerlerinde test yapılmasını amaçlayan test tekniği.
Edge Case (Uç Durum)	Normal kullanımın dışında kalan sınır veya ekstrem durumlar.
Mock	Gerçek bağımlılık yerine test amacıyla kullanılan sahte nesne.
Mocking	Testlerde dış bağımlılıkların izole edilmesi için sahte nesnelerin kullanılması.
Test Coverage (Kapsam Oranı)	Testlerin kodun ne kadarını çalıştırdığını gösteren ölçüm.
Line Coverage	Kod satırlarının testler tarafından çalıştırılma oranı.

Branch Coverage	Koşullu yapıların (if/else) test edilme oranı.
Method Coverage	Metotların test edilme oranı.
Test Execution Raporu	Testlerin çalıştırılması sonucu oluşan başarı/başarısızlık raporu.
Defect (Hata)	Test sürecinde tespit edilen beklenmeyen veya hatalı sistem davranışı.
Defect Log	Tespit edilen hataların detaylarıyla kayıt altına alındığı tablo.
Severity	Bir hatanın sistem üzerindeki etki derecesi.
Known Issue (Bilinen Hata)	Çözümü sonraya bırakılmış, dokümante edilmiş hata.
Flaky Test	Aynı koşullarda bazen geçen bazen kalan kararsız test.
Test Ortamı	Testlerin gerçek sistemden izole şekilde çalıştırıldığı ortam.
In-Memory Database	Verilerin disk yerine RAM üzerinde tutulduğu geçici veritabanı yapısı.
Code Coverage Raporu	Kod kapsama oranlarının araçlar aracılığıyla raporlanmış hali.
Kalite Metriği	Yazılımın güvenilirlik, bakım yapılabilirlik ve izlenebilirlik gibi özelliklerini ölçen kriter.
Traceability (İzlenebilirlik)	Test senaryolarının ilgili gereksinimlerle birebir ilişkilendirilebilmesi.

7. Kaynakça

1- https://start.spring.io/

2- https://antigravity.google/

3- https://dev.mysql.com/downloads/workbench/

4-https://maven.apache.org/

5-https://www.java.com/en/download/manual.jsp